

Correttezza di Programmi e Cicli

Progettazione dei cicli e Invarianti di ciclo

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

Correttezza di un Programma

Correttezza di un Programma



**Correttezza di un
programma =**

Correttezza parziale +

Terminazione

Perché la correttezza è divisa in due parti? v



Perché le **metodologie** per garantire le due parti sono **distinte**

Definizioni: Preliminari



- Dato una funzione `prog` tale che:

```
/** @brief Dato l'input  $x_1, \dots, x_n$  ( $n \geq 0$ ) prog restituisce l'output  $z$  */
```

```
t prog(t1 x1, ..., tn xn) {  
    /* CORPO DELLA FUNZIONE */  
}
```

- E due condizioni:

- $P\text{-IN}(x_1, \dots, x_n)$: una condizione su $\mathbf{x}_1, \dots, \mathbf{x}_n$
 - Detta **predicato di input** (P-IN)
- $P\text{-OUT}(x_1, \dots, x_n, z)$: una condizione su $\mathbf{x}_1, \dots, \mathbf{x}_n$ e $\mathbf{z}$
 - Detta **predicato di output** (P-OUT)

Definizioni: Correttezza Parziale



CORRETTEZZA PARZIALE (rispetto a P-IN e P-OUT)

- `prog` è **parzialmente corretta** quando:
 - Se per tutti i valori x_1 di tipo t_1 , ..., x_n di tipo t_n
 - Vale $P\text{-IN}(x_1, \dots, x_n)$
 - E l'esecuzione di `prog( $t_1$   $x_1$ , ...,  $t_n$   $x_n$ )` termina restituendo z
 - Allora
 - Vale $P\text{-OUT}(x_1, \dots, x_n, z)$

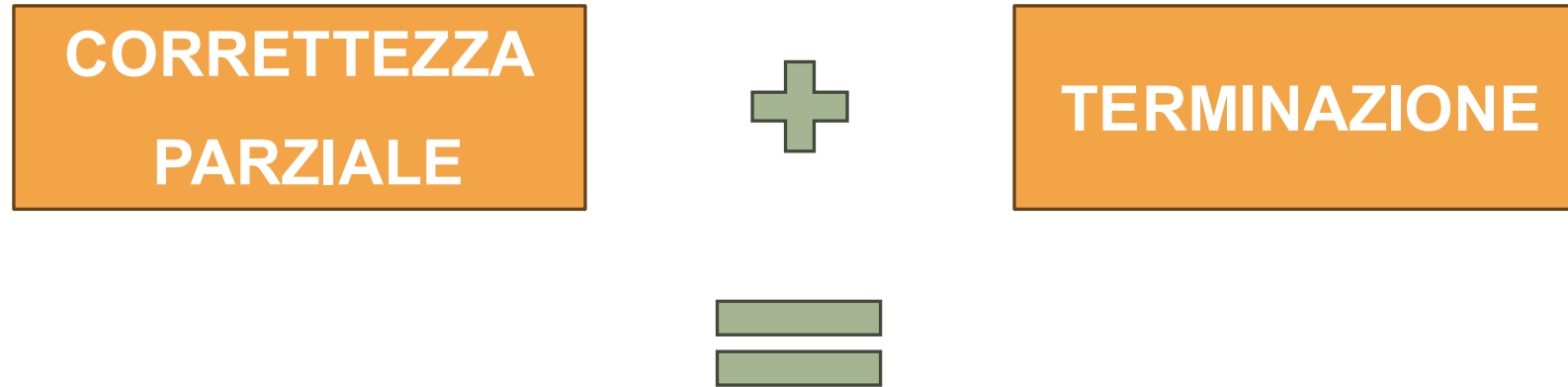
Definizioni: Terminazione



TERMINAZIONE (rispetto a P-IN)

- `prog` è **terminante**
 - Se per tutti i valori x_1 di tipo t_1 , ..., x_n di tipo t_n
 - Vale $P\text{-}IN(x_1, \dots, x_n)$
 - Allora
 - L'esecuzione di `prog( $t_1$   $x_1$ , ...,  $t_n$   $x_n$ )` termina

Definizioni: Correttezza



CORRETTEZZA (rispetto a P-IN e P-OUT)

- prog è **corretto** se è:
 - **parzialmente corretto**
 - **terminante**

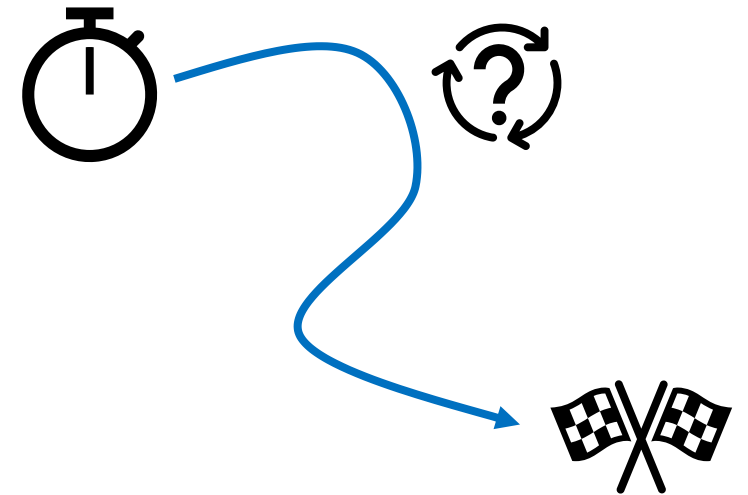
Come si Dimostra la Correttezza?

- Due metodologie:
 - **Post-hoc Verification (PhV)**
 - Prima vengono creati una specifica (i.e., pre/postcondizione) e un programma. Successivamente, viene verificato che il programma soddisfi la specifica
 - **Correctness-by-Construction (CbC)**
 - I programmi sono costruiti utilizzando regole di raffinamento che garantiscono che l'implementazione risultante sia corretta rispetto ad una data specifica (pre/postcondizione).
- Nelle prossime slides vedremo una metodologia **CbC** per programmi iterativi:
 - METODOLOGIA DI SVILUPPO **GUIDATA DALL'INVARIANTE DI CICLO**

Sviluppo guidato dall'Invariante di Ciclo

Invariante di Ciclo

- Per scrivere un ciclo non banale è indispensabile pensare subito al passo generico
 - Non al passo iniziale
- Cioè:
 - Bisogna pensare **non** agli estremi (inizio o fine del ciclo)
 - Bensì ad un **generico punto intermedio**
- **Invariante di ciclo:**
 - La generica situazione in un punto intermedio, opportunamente precisata, costituisce ciò che si chiama l'**invariante del ciclo**.



Sviluppo di un Ciclo Corretto per Costruzione I



- Generico ciclo iterativo costituito da tre parti:

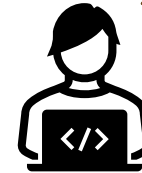
inizializzazione

while(*condizione*)

corpo dell'iterazione



NO



L'ordine giusto è l'opposto!
(almeno in linea di principio)

- Per scrivere l'inizializzazione
 - Si deve sapere cosa deve fare il ciclo
- Per scrivere la condizione (guardia)
 - Si deve conoscere cosa farà il corpo

Sviluppo di un Ciclo Corretto per Costruzione II



- Si concepisce il programma nell'ordine inverso a quello della sua forma finale.
- Si scrivono nell'ordine:
 1. il corpo del ciclo
 2. la condizione di controllo del ciclo
 3. l'inizializzazione

Infatti

- Prima si concepiscono le istruzioni del corpo del ciclo
 - In modo che producano un avvicinamento alla soluzione
- Poi si scrive il test di controllo del ciclo
 - In modo che all'uscita dal ciclo si abbia la soluzione corretta
- Per ultimo si scrive l'inizializzazione
 - In modo da garantire una "partenza corretta"

Primo Esempio

Esempio: Calcolo del Massimo in un Array I

```
/** @brief Assume: lunghezza(a) >= n >= 1 P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce:
 * - il massimo elemento che occorre nella porzione di array a[0..n-1].
 *   P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
 */
int maximum(const int[] a, int n);
```

File maximum.h

Applichiamo lo sviluppo guidato dall'invariante di ciclo:

1. Per prima cosa definiamo i predicati di input e output

- P-IN e P-OUT
- e li aggiungiamo alla descrizione
 - sia della dichiarazione della funzione (file .h)
 - che all'implementazione della funzione (file .c)
- P-IN e P-OUT specificano **COSA** si deve fare

P-IN?

P-IN(**a**, **n**): lunghezza(**a**) ≥ **n** ≥ 1

P-OUT?

P-OUT(**a**, **n**, **r**): **r** è il massimo in **a**[0,..., **n**-1]

Esempio: Calcolo del Massimo in un Array II

```
/** @brief Assume: lunghezza(a) >= n >= 1 P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce:
 * - il massimo elemento che occorre nella porzione di array a[0..n-1].
 *   P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
 */
int maximum(const int[] a, int n){

}
```

File `maximum.c`

2. Poi si identifica un insieme “significativo” di test

- Potrà essere esteso durante lo sviluppo
- Organizzati in una **test suite**
 - In realtà, questo passo non fa parte della metodologia di sviluppo guidata dall'invariante di ciclo, però pensare prima ai test aiuta a scrivere un codice migliore e a farlo più rapidamente

Esempio: Memento Testing

Prima di iniziare a sviluppare il codice, individuate casi (i più semplici possibili) che descrivano le varie tipologie di input e scrivete una test suite che, in generale, estenderete durante lo sviluppo.

```
// Test suite per il file maximum.c
#include <stdio.h>
#include "maximum.h"

int main(void) {
    puts("BEGIN TEST maximum.c");
    /* IMPORTANTE: Ogni blocco codifica un
       UNIT TEST isolato dagli altri */
    int n = 1; int a[n] = {3 };
    int maxExpected = 3;
    int max = maximum(a,n);
    if (max != maxExpected) {
        printf("--TEST 1 FAILED--\n");
        printf("EXPECTED:%d\n", maxExpected);
        printf("RESULT:%d\n\n", max);}
}

/* Per brevità, ometto il codice per gli
   UNIT TESTS di seguenti 10 (input,output):
* ({1, 3}, 2), 3)
* ({3, 1}, 2), 3)
* ({3, 3}, 3)
* ({1, 2, 3, 5}, 4), 5)
* ({5, 1, 2, 3}, 4), 5)
* ({1, 5, 2, 3}, 4), 5)
* ({1, 2, 4, 3}, 4), 5)
* ({5, 5, 3, 2}, 4), 5)
* ({3, 1, 5, 5}, 4), 5)
* ({5, 5, 5, 5}, 4), 5) */
}
```

Esempio: Calcolo del Massimo in un Array III



3. Poi si scrive un breve testo:

- Descrive senza fornire tutti i dettagli l'idea alla base del procedimento da codificare
- Elenca i passi da dettagliare –
- Cioè si dichiara, in modo non dettagliato:
 - **COME** si pensa di fare quello che si deve fare

ISPEZIONEREMO GLI ELEMENTI DELL'ARRAY a A PARTIRE DALLA POSIZIONE 0 FINO ALLA POSIZIONE $n-1$
CONSIDERIAMO LA SITUAZIONE IN CUI:

– LA VARIABILE max CONTIENE IL MASSIMO DI $a[0..i-1]$ ($1 \leq i \leq n-1$);

CHE COSA DOBBIAMO FARE PER:

- (1) AVVICINARCI ALLA SOLUZIONE ?
- (2) CONTROLLARE SE SIAMO ARRIVATI ALLA SOLUZIONE ?
- (3) INIZIARE A ISPEZIONARE a ?

Esempio: Calcolo del Massimo in un Array IV



```
/** @brief Assume: lunghezza(a) >= n >= 1 P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce: il massimo elemento che occorre nella porzione di array a[0..n-1].
 * P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
 */
int maximum(const int[] a, int n){
    int max = /* VALORE INIZIALE */
    // I.C.: max è il massimo in a[0..i-1]
    for (int i = /* INDICE INIZIALE */ ; /* CONDIZIONE */; /* INCREMENTO DELL'INDICE */) {
        /* CORPO DEL CICLO */
    }
    return max;
}
```

File maximum.c

4. Poi si scrivono:

- Il template del corpo della funzione (che deve contenere un ciclo)
- L'invariante di ciclo (I.C.)
- E si procede nello sviluppo assumendo che I.C. valga
 - Se (nel corso dello sviluppo) questa assunzione si rivelasse impraticabile (ovvero portasse a inconsistenze che non si sa risolvere), allora la si modificherà

Esempio: Calcolo del Massimo in un Array V

```
P-IN(a, n): lunghezza(a) ≥ n ≥ 1    P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
int maximum(const int[] a, int n){
    int max = /* VALORE INIZIALE */
    // I.C.: max è il massimo in a[0..i-1] <===== l'aggiornamento i++ ripristina l'I.C.
    for (int i = /* INDICE INIZIALE */ ; i < n; i++) {    <===== (I.C. ∧ (i== n)) → P-OUT(a,n,max)
        if (max < a[i]) max = a[i]; <===== max è il massimo in a[0..(i+1)-1]
    }
    return max; <===== P-OUT(a,n,max) ≡ max è il massimo in a[0..n-1]
}
```

File maximum.c

5. Poi si espandono in ordine:

- Il corpo del ciclo
 - In modo tale che la sua esecuzione faccia avanzare il calcolo del risultato
- L'aggiornamento delle variabili coinvolte nella condizione del ciclo
 - In modo che ripristini l'invariante di ciclo
 - Ovvero tale che l'invariante di ciclo sia vero alla fine di ogni singola iterazione del ciclo
- La condizione del ciclo
 - In modo che dall'invariante di ciclo e dalla negazione della condizione si possa dedurre la terminazione del calcolo

Esempio: Calcolo del Massimo in un Array VI



```
P-IN(a, n): lunghezza(a) ≥ n ≥ 1    P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
int maximum(const int[] a, int n){
    int max = a[0];
    // I.C.: max è il massimo in a[0..i-1] <===== l'aggiornamento i++ ripristina l'I.C.
    for (int i = 1 ; i < n; i++) {    <===== (I.C. ∧ (i== n)) → P-OUT(a,n,max)
        if (max < a[i]) max = a[i]; <===== max è il massimo in a[0..(i+1)-1]
    }
    return max; <===== P-OUT(a,n,max) ≡ max è il massimo in a[0..n-1]
}
```

File maximum.c

6. Infine si scrivono

- I valori iniziali delle variabili coinvolte nel ciclo
 - In modo tale che l'invariante di ciclo sia vera all'inizio della prima iterazione
- A questo punto si è di fatto dimostrata la CORRETTEZZA PARZIALE della funzione
 - Se P-IN vale e maximum() restituisce r, allora P-OUT vale?

Finito?

No, manca dimostrazione della terminazione

Esempio: Calcolo del Massimo in un Array VII



```
P-IN(a, n): lunghezza(a) ≥ n ≥ 1    P-OUT(a, n, r): r è il massimo in a[0,..., n-1]
int maximum(const int[] a, int n){
    int max = a[0];
    // I.C.: max è il massimo in a[0..i-1] <===== l'aggiornamento i++ ripristina l'I.C.
    for (int i = 1 ; i < n; i++) {    <===== (I.C. ∧ (i== n)) → P-OUT(a,n,max)
        if (max < a[i]) max = a[i]; <===== max è il massimo in a[0..(i+1)-1]
    }
    return max; <===== P-OUT(a,n,max) ≡ max è il massimo in a[0..n-1]
}
```

File maximum.c

- Dimostrazione della TERMINAZIONE
 - Ad ogni iterazione del ciclo il valore di i aumenta di 1, mentre il valore di n non cambia
 - Quindi in un numero finito di iterazioni il valore di i raggiungerà il valore n e il ciclo terminerà

Secondo Esempio

Esempio: Ultima Occorrenza del Massimo I

```
/** @brief Assume: lunghezza(a) >= n >= 1. P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo nella porzione a[0..n-1].
 */ P-OUT(a, n, r): r è l'indice dell'ultima occorrenza del massimo in a[0,...,n-1]
int findLastMax(const int[] a, int n);
```

File findLastMax.h

Applichiamo lo sviluppo guidato dall'invariante di ciclo:

1. Per prima cosa definiamo i predicati di input e output

- P-IN e P-OUT
- e li aggiungiamo alla descrizione
 - sia della dichiarazione della funzione (file .h)
 - che all'implementazione della funzione (file .c)
- P-IN e P-OUT specificano **COSA** si deve fare

P-IN?

P-IN(**a**, **n**): lunghezza(**a**) ≥ **n** ≥ 1

P-OUT?

P-OUT(**a**, **n**, **r**): **r** è l'indice dell'ultima occorrenza del massimo in **a**[0,...,**n**-1]

Esempio: Ultima Occorrenza del Massimo II

```
/** @brief Assume: lunghezza(a) >= n >= 1. P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo nella porzione a[0..n-1].
 */ P-OUT(a, n, r): r è l'indice dell'ultima occorrenza del massimo in a[0,...,n-1]
int findLastMax(const int[] a, int n) {

}
```

File findLastMax.c

2. Poi si identifica un insieme “significativo” di test

- Potrà essere esteso durante lo sviluppo
- Organizzati in una **test suite**
 - Ricordo che in realtà, questo passo non fa parte della metodologia di sviluppo guidata dall'invariante di ciclo, però pensare prima ai test aiuta a scrivere un codice migliore e a farlo più rapidamente

Esempio: Memento Testing

Prima di iniziare a sviluppare il codice – anche se non volete applicare alla lettera la metodologia TDD – individuate casi (il più semplici possibile) che descrivano le varie tipologie di input e scrivete una test suite che, in generale, estenderete durante lo sviluppo.

```
// Test suite per il file findLastMax.c
#include <stdio.h>
#include "findLastMax.h"
void check(int t, int a[], int n, int re) {
    int r = findLastMax(a,n);
    if (r != re) {
        printf("--TEST %d FAILED--\n", t);
        printf("EXPECTED:%d\n", re);
        printf("RESULT:%d\n\n", r);
    }
}
int main(void) {
    puts("BEGIN TEST findLastMax.c");
    /* IMPORTANTE: Ogni blocco codifica un
       UNIT TEST isolato dagli altri */
    int n = 1; int a[n] = {3 }; int re = 0;
    check(1, a, n, re);
}
/* Per brevità, ometto il codice per gli
   UNIT TESTS di seguenti 10 (input,output):
   * ({1, 3}, 2), 1)
   * ({3, 1}, 2), 0)
   * ({3, 3}, 1)
   * ({1, 2, 3, 5}, 4), 3)
   * ({5, 1, 2, 3}, 4), 0)
   * ({1, 5, 2, 3}, 4), 1)
   * ({1, 2, 4, 3}, 4), 2)
   * ({5, 5, 3, 2}, 4), 1)
   * ({3, 1, 5, 5}, 4), 3)
   * ({5, 5, 5, 5}, 4), 3) */
}
```

Esempio: Ultima Occorrenza del Massimo III

3. Poi si scrive un breve testo:

- Descrive senza fornire tutti i dettagli l'idea alla base del procedimento da codificare
- Elenca i passi da dettagliare –
- Cioè si dichiara, in modo non dettagliato:
 - **COME** si pensa di fare quello che si deve fare

ISPEZIONEREMO GLI ELEMENTI DELL'ARRAY a A PARTIRE DALLA POSIZIONE 0 FINO ALLA POSIZIONE $n-1$
CONSIDERIAMO LA SITUAZIONE IN CUI:

i_{Max} CONTIENE L'INDICE DELL'ULTIMA OCCORRENZA DEL MASSIMO IN $a[0..i-1]$ ($1 \leq i \leq n-1$)

CHE COSA DOBBIAMO FARE PER:

- (1) AVVICINARCI ALLA SOLUZIONE ?
- (2) CONTROLLARE SE SIAMO ARRIVATI ALLA SOLUZIONE ?
- (3) INIZIARE A ISPEZIONARE a ?

Esempio: Ultima Occorrenza del Massimo IV

```
/** @brief Assume: lunghezza(a) >= n >= 1. P-IN(a, n): lunghezza(a) ≥ n ≥ 1
 * Restituisce:
 * - l'indice dell'ultima occorrenza del massimo nella porzione a[0..n-1].
 */ P-OUT(a, n, r): r è l'indice dell'ultima occorrenza del massimo in a[0,...,n-1]
int findLastMax(const int[] a, int n) {
    int iMax = /* VALORE INIZIALE */;
    // I.C.: iMax è l'indice dell'ultima occorrenza del massimo in a[0..i-1]
    for (int i = /* INDICE INIZIALE */ ; /* CONDIZIONE */; /* INCREMENTO DELL'INDICE */) {
        /* CORPO DEL CICLO */
    }
    return iMax;
}
```

File findLastMax.c

3. Poi si scrivono:

- Il template del corpo della funzione (che deve contenere un ciclo)
- L'invariante di ciclo (I.C.)
- E si procede nello sviluppo assumendo che I.C. valga
 - Se (nel corso dello sviluppo) questa assunzione si rivelasse impraticabile (ovvero portasse a inconsistenze che non si sa risolvere), allora la si modificherà

Esempio: Ultima Occorrenza del Massimo V

P-IN(**a**, **n**): lunghezza(**a**) ≥ **n** ≥ 1 P-OUT(**a**, **n**, **r**): **r** è l'indice ultima occorrenza del massimo in **a**[0,...,**n**-1]

```
int findLastMax(const int[] a, int n) {  
    int iMax = /* VALORE INIZIALE */;  
    // I.C.: iMax è l'indice ultima occorrenza massimo in a[0..i-1] ⇐ l'aggiornamento i++ ripristina l'I.C.  
    for (int i = /* INDICE INIZIALE */ ; i < n; i++) {    ⇐==== (I.C. ∧ (i== n)) → P-OUT(a,n,iMax)  
        if (a[iMax] <= a[i]) iMax = i; ⇐==== iMax è l'indice ultima occorrenza del massimo in a[0..(i+1)-1]  
    }  
    return iMax;    ⇐===== P-OUT(a,n,iMax) ≡ iMax è l'indice ultima occorrenza del massimo in a[0..n-1]  
}
```

File findLastMax.c

4. Poi si espandono in ordine:

- Il corpo del ciclo
 - In modo tale che la sua esecuzione faccia avanzare il calcolo del risultato
- L'aggiornamento delle variabili coinvolte nella condizione del ciclo
 - In modo che ripristini l'invariante di ciclo
 - Ovvero tale che l'invariante di ciclo sia vero alla fine di ogni singola iterazione del ciclo
- La condizione del ciclo
 - In modo che dall'invariante di ciclo e dalla negazione della condizione si possa dedurre la terminazione del calcolo

Esempio: Ultima Occorrenza del Massimo VI

```
P-IN(a, n): lunghezza(a) ≥ n ≥ 1  P-OUT(a, n, r): r è l'indice ultima occorrenza del massimo in a[0,...,n-1]
int findLastMax(const int[] a, int n) {
    int iMax = 0;
    // I.C.: iMax è l'indice ultima occorrenza massimo in a[0..i-1] ⇐ l'aggiornamento i++ ripristina l'I.C.
    for (int i = 1; i < n; i++) {  ⇐==== (I.C. ∧ (i == n)) → P-OUT(a,n,iMax)
        if (a[iMax] <= a[i]) iMax = i; ⇐==== iMax è l'indice ultima occorrenza del massimo in a[0..(i+1)-1]
    }
    return iMax;  ⇐===== P-OUT(a,n,iMax) ≡ iMax è l'indice ultima occorrenza del massimo in a[0..n-1]
}
```

File findLastMax.c

6. Infine si scrivono

- I valori iniziali delle variabili coinvolte nel ciclo
 - In modo tale che l'invariante di ciclo sia vera all'inizio della prima iterazione
- A questo punto si è di fatto dimostrata la CORRETTEZZA PARZIALE della funzione
 - Se P-IN vale e findLastMax() restituisce **r**, allora P-OUT vale?

Finito?

No, manca dimostrazione della terminazione

Esempio: Ultima Occorrenza del Massimo VII



```
P-IN(a, n): lunghezza(a) ≥ n ≥ 1  P-OUT(a, n, r): r è l'indice ultima occorrenza del massimo in a[0,...,n-1]
int findLastMax(const int[] a, int n) {
    int iMax = 0;
    // I.C.: iMax è l'indice ultima occorrenza massimo in a[0..i-1] ⇐ l'aggiornamento i++ ripristina l'I.C.
    for (int i = 1; i < n; i++) { ⇐==== (I.C. ∧ (i== n)) → P-OUT(a,n,iMax)
        if (a[iMax] <= a[i]) iMax = i; ⇐==== iMax è l'indice ultima occorrenza del massimo in a[0..(i+1)-1]
    }
    return iMax; ⇐===== P-OUT(a,n,iMax) ≡ iMax è l'indice ultima occorrenza del massimo in a[0..n-1]
}
```

File findLastMax.c

- Dimostrazione della TERMINAZIONE
 - Ad ogni iterazione del ciclo il valore di **i** aumenta di 1, mentre il valore di **n** non cambia
 - Quindi in un numero finito di iterazioni il valore di **i** raggiungerà il valore **n** e il ciclo terminerà

Notazioni e Convenzioni

Convenzioni e Notazioni I



- I due esempi visti rispettano lo schema:

```
/** @brief Dato l'input  $x_1, \dots, x_n$  ( $n \geq 0$ ) restituisce l'output  $z$   
* P-IN( $x_1, \dots, x_n$ ): una condizione su  $x_1, \dots, x_n$  (detta "predicato di input")  
* P-OUT( $x_1, \dots, x_n, z$ ): una condizione su  $x_1, \dots, x_n$  e  $z$  (detta "predicato di output")  
*/  
t prog( $t_1 x_1, \dots, t_n x_n$ ) { /* CORPO DELLA FUNZIONE */ }
```

- Dove, i valori di $x_1, \dots, x_n$ non sono modificati nel corpo della funzione
- In pratica è comune scrivere funzioni
 - In cui i valori di qualche x_i ($1 \leq i \leq n$) sono modificati
 - z non sempre c'è
- In questi casi, nel predicato di output si scrive
 - $x'_1, \dots, x'_n$ per riferirsi ai valori iniziali delle variabili $x_1, \dots, x_n$

Convenzioni e Notazioni II



- In particolare:
 - Se x_i è un array, allora si scrive x'_i per riferirsi al valore iniziale dei suoi elementi
 - Se x_i è un puntatore, allora si scrive $(*x_i)'$ per riferirsi al valore iniziale di $(*x_i)$
- Ad esempio:

```
/** @brief Scambia due elementi in un array
 * P-IN(a,first,second): first e second sono indici
validi di a
 * P-OUT(a,first,second): a[first] == a'[second] e
a[second] == a'[first]
 */
void swap(int[] a, int first, int second) {
    int hold = a[first];
    a[first] = a[second];
    a[second] = hold;
}
```

```
/** @brief Scambia il contenuto di due variabili
 * P-IN(px,py): true
 * P-OUT(px,py): *px == (*py)' e *py == (*px)'
 */
void swap2(int *px , int *py) {
    int hold = *px;
    *px = *py;
    *py = hold;
}
```

TDD vs CbC vs PhV

TDD vs CbC/PhV



- TDD: una **Test Suite** è una specifica:
 - **Formale** (è scritta in linguaggio di programmazione)
 - **Precisa** (al più quanto lo è la specifica del linguaggio in cui è scritta)
 - **Estensionale** (descrive delle coppie $\langle \text{input}, \text{output} \rangle$)
 - **Incompleta** (salvo casi triviali, sarà sempre incompleta)
 - **Significativa** (deve descrivere dei casi interessanti)
 - **Verificabile** in modo automatico (i test sono eseguibili)
 - **In evoluzione** (guida lo sviluppo e l'evoluzione del codice per iterazioni successive)
- CbC/PhV: una coppia $\langle \text{P-IN}(\dots), \text{P-OUT}(\dots) \rangle$ è una specifica:
 - **Formale** (è scritta in un formalismo logico)
 - **Precisa** (un formalismo logico, per essere tale, deve essere preciso)
 - **INTENSIONALE** (descrive la relazione tra input e output)
 - **COMPLETA** (di norma lo deve essere)
 - **Significativa** (deve descrivere una relazione input-output interessante)
 - **Verificabile** in modo SEMI-automatico (per alcune coppie logica-linguaggio esistono dei tools, idealmente il programmatore si limita a scrivere la specifica $\langle \text{PIN-IN}(\dots), \text{P-OUT}(\dots) \rangle$ e gli Invarianti di Ciclo)
 - **In evoluzione** (se il codice della funzione deve evolvere, allora l'evoluzione della specifica guida l'evoluzione del codice della funzione)

Commenti Finali

- La metodologia di sviluppo guidato dall'invariante di ciclo è un approccio sistematico per sviluppare correttamente programmi (per noi, funzioni C) il cui corpo è costituito da uno o più cicli
 - Dovrebbe sempre essere utilizzata quando si scrive un ciclo non banale
 - È un approccio complementare (non alternativo) alla metodologia di sviluppo guidata dal testing
- In pratica la metodologia più utilizzata è:
 - TDD in combinazione con Regression Testing e riutilizzo del codice
 - Utilizzando tools di supporto (che sono relativamente semplici da usare in modo efficace)
- Tuttavia per applicazioni “safety critical” si utilizza anche:
 - La metodologia di sviluppo guidato dall'invariante di ciclo
 - Utilizzando tools di supporto (che richiedono però una preparazione elevata per usati in modo efficace)
- Un informatico deve padroneggiare argomenti come la metodologia di sviluppo guidato dall'invariante di ciclo
 - Saper ragionare sul codice è necessario per avvalersi di:
 - Riutilizzo del codice, TDD, “low code”, LLM, ...

